

Rapport final de projet de base de données et web : The Wave

Bastien BONNE - Mouatizbillah BELBAGRA

Introduction et présentation du projet

Présentation générale du projet

Le projet **The Wave** est une plateforme de streaming musical et une base de données relationnelle dont l'objectif principal est de modéliser l'écosystème complexe de l'industrie musicale en ligne. Ce système permet de gérer les artistes, les groupes, les albums, les morceaux, les utilisateurs, les playlists et l'historique d'écoute.

L'objectif fonctionnel est de fournir une infrastructure de données solide pour une application web qui permettrait :

1. L'écoute et l'enregistrement de l'historique des morceaux.
2. La gestion des playlists personnelles et publiques.
3. Le suivi d'autres utilisateurs et de groupes.
4. L'établissement d'un système de redevances estimées pour les artistes et groupes.

Le projet a été développé en se concentrant sur les technologies de base de données relationnelles, notamment **PostgreSQL**, pour sa robustesse et sa gestion avancée des contraintes et des vues.

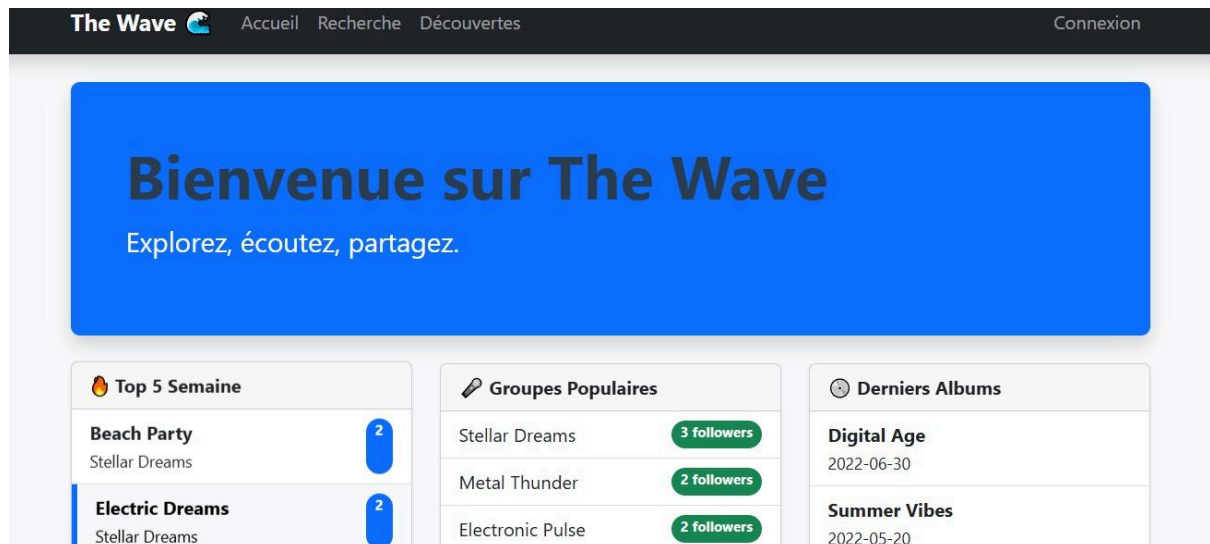
Petit manuel utilisateur

Le site The Wave s'adresse aux mélomanes souhaitant organiser leur expérience d'écoute.

Scénario : découverte et personnalisation

1. L'**utilisateur** se connecte à son compte.
2. Sur la page d'accueil, il accède à la section « **Tendances** » qui affiche les morceaux les plus écoutés du moment.
3. Il lance l'écoute d'un nouveau morceau du groupe *Stellar Dreams*. L'action d'écoute est automatiquement enregistrée dans la base de données.
4. Il décide de suivre ce groupe et son créateur via la fonctionnalité de *follow*.

5. Il découvre une playlist publique intéressante créée par un utilisateur qu'il suit, puis ajoute deux morceaux de cette playlist à sa propre collection personnelle.



Capture d'écran de l'interface générale du site The Wave

Section technique : synthèse des étapes 1 et 2

Architecture et modélisation (Étape 1)

L'architecture du projet repose sur un modèle classique Client-Serveur, où le cœur du système est la base de données relationnelle.

La conception a été formalisée via un Modèle Entité-Association détaillé. Ce modèle met en lumière les entités fondamentales :

- **Utilisateur** (avec **pseudo**, **email**, **mdp** sécurisé).
- **Groupe** et **Artiste** (liés via l'association **appartient** et **participe**).
- **Morceau** et **Album** (Morceau a une dépendance forte à Album et Groupe).
- **Playlist** (liée à Morceau via **possede**).
- **Record historique** (pour les écoutes, garantissant l'enregistrement de l'instant précis).

Le modèle gère des relations complexes telles que : la relation **N-N** entre **utilisateur** et **utilisateur** (via **follow**), entre **utilisateur** et **groupe** (via **suit**), et entre **groupe** et **artiste** (via **appartient**)

Modèle relationnel et contraintes (Étape 2)

Le passage au modèle relationnel a conduit à la création de 14 tables, en respectant les règles de l'algèbre relationnelle et en intégrant des contraintes pour assurer l'intégrité et la cohérence des données.

Contraintes d'intégrité clés

Pour garantir la qualité des données, des contraintes spécifiques ont été définies :

Table	Contrainte	Objectif
artiste	CHECK(date_M > date_N)	Assurer qu'une date de mort est postérieure à la date de naissance.
utilisateur	CHECK(date_N <= CURRENT_DATE - INTERVAL '13 years')	Exiger un âge minimal de 13 ans pour l'inscription.
morceau	UNIQUE(idal, positionA)	Garantir qu'un morceau a une position unique au sein de son album.
morceau	CHECK (duree > '00:00:00')	S'assurer que chaque morceau a une durée positive.

Description technique des fonctionnalités et requêtes SQL

Cette section présente trois fonctionnalités essentielles du site The Wave, en détaillant l'utilisation des requêtes SQL pour interagir avec la base de données.

Fonctionnalité : afficher le top 5 des morceaux du moment

Cette fonctionnalité permet de mettre en avant les morceaux les plus populaires en se basant sur le nombre d'écoutes totales. Elle exploite une **VIEW** pour l'agrégation et une jointure pour associer le morceau à son groupe.

Objectif	Afficher le titre du morceau, le nom du groupe et le nombre total d'écoutes, classé du plus écouté au moins écouté.
Utilité	Proposer une page d'accueil dynamique ou une section "Tendances".

Requête SQL:

```
WITH nb_ecoutes AS (  
  SELECT  
    idm,  
    COUNT(*) AS total_ecoutes  
  FROM  
    record_historique  
  GROUP BY  
    idm  
)  
  
SELECT  
  M.titre,  
  G.nomg AS Nom_du_Groupe,  
  E.count AS Nombre_Total_Ecoutes  
FROM  
  morceau M  
JOIN  
  groupe G ON M.idg = G.idg  
JOIN  
  nb_ecoutes E ON M.idm = E.idm  
ORDER BY  
  Nombre_Total_Ecoutes DESC  
LIMIT  
  5;
```

Fonctionnement et utilité : La vue `nb_ecoutes` est utilisée pour pré-calculer le nombre d'écoutes par morceau. La requête effectue ensuite une double jointure pour relier ces statistiques au titre du morceau (`morceau M`) et au nom du groupe (`groupe G`). L'instruction `ORDER BY ... DESC LIMIT 5` assure que seuls les cinq morceaux les plus écoutés sont retournés, garantissant une réponse rapide aux requêtes d'affichage de tendances.

Fonctionnalité : calculer le revenu estimé d'un groupe

Cruciale pour l'aspect commercial, cette fonctionnalité calcule les redevances dues à un groupe selon une logique métier prédéfinie : la première écoute d'un morceau par un utilisateur donné génère un revenu supérieur (0,10€) aux écoutes suivantes (0,01€).

Objectif	Calculer la rémunération totale pour un groupe en différenciant les premières écoutes uniques des écoutes répétées.
Utilité	Fonctionnalité de back-office pour la gestion des droits et le reporting financier.

Requête SQL (via la vue agrégée `revenue_groupes`)

```
SELECT
    G.nomg,
    R.revenue
FROM
    groupe G
JOIN
    revenue_groupes R ON G.idg = R.idg
WHERE
    G.nomg = 'NOM_DU_GROUPE_RECHERCHE' ;
```

Fonctionnement et utilité : La complexité du calcul est encapsulée dans la vue `revenue_groupes`. Cette vue utilise des fonctions d'agrégation (`COUNT`) et la clause `DISTINCT` sur la paire (`idu`, `idm`) pour identifier les premières écoutes. Ce mécanisme permet de désolidariser la complexité du calcul du revenu du code applicatif, assurant une logique de rémunération cohérente et facilement modifiable au niveau de la base de données.

Fonctionnalité : afficher les playlists publiques créées par les utilisateurs suivis

Cette requête illustre la gestion des relations sociales complexes au sein de la base de données.

Objectif	Lister le titre des playlists publiques créées par les utilisateurs que l'utilisateur connecté suit.
Utilité	Créer une section de découverte sociale de contenu pour l'utilisateur.

Requête SQL

```
SELECT
  P.titre AS Titre_Playlist,
  U2.pseudo AS Createur_Playlist
FROM
  follow F
JOIN
  utilisateur U1 ON F.idu = U1.idu
JOIN
  utilisateur U2 ON F.idu2 = U2.idu  -- U2 est l'utilisateur suivi (créateur)
JOIN
  playlist P ON U2.idu = P.idu
WHERE
  F.idu = ID_UTILISATEUR_CONNECTE
  AND P.statut = 'public';
```

Fonctionnement et utilité : La requête requiert une auto-jointure sur la table **utilisateur** (via la table **follow**) pour distinguer l'utilisateur suiveur (**U1**) de l'utilisateur suivi (**U2**). Elle filtre ensuite les résultats pour ne retenir que les playlists dont le statut est explicitement '**public**', assurant que la confidentialité des playlists privées est respectée.

Pistes d'améliorations

Le projet The Wave, bien que solide en termes de modélisation BDD, présente des pistes d'amélioration significatives pour une mise en production :

Améliorations de sécurité et performance

- **Gestion des mots de passe** : Le champ `mdp` est actuellement un `VARCHAR(25)` et doit impérativement être remplacé par un champ de type `CHAR(60)` ou `TEXT` pour stocker des **empreintes de hachage sécurisées** (ex: Argon2, bcrypt) et non le mot de passe en clair.
- **Indexation manuelle (performance)** : Ajouter des index sur les clés étrangères des tables très sollicitées, comme `idm` et `idu` dans `record_historique`, pour accélérer la recherche et les agrégations.
- **Gestion de l'entité historique** : La colonne `historique` dans `record_historique` est stockée comme un `timestamp with time zone`. Bien que précis, il faudrait ajouter un index composite sur (`idu`, `idm`, `historique`) pour optimiser les requêtes qui cherchent à déterminer la première écoute d'un utilisateur.

Améliorations fonctionnelles

- **Système de notation (Avis)** : Ajouter une table N-N `note` ou `avis` (entre `utilisateur` et `morceau` ou `album`) pour permettre aux utilisateurs de laisser des notes, ce qui permettrait d'enrichir les tendances (au-delà du simple nombre d'écoutes).
- **Gestion des contrats avancée** : Mettre en place un mécanisme de gestion des périodes de validité des contrats. L'utilisation d'une contrainte `CHECK(date_sortie > date_entree)` est un bon début, mais il faudrait ajouter un filtre côté application pour ne considérer que les contrats actifs (`date_entree <= CURRENT_DATE AND date_sortie >= CURRENT_DATE`).

Organisation et Répartition du Travail

Le projet a été réalisé en binôme par Bastien BONNE et Mouatizbillah BELBAGRA. Une répartition claire des tâches a été essentielle pour la progression du travail.

Membre du Binôme	Rôles et Responsabilités Principales
Bastien BONNE	Conception du Schéma Entité-Association, Définition du Modèle Relationnel, Définition des Contraintes d'Intégrité.
Mouatizbillah BELBAGRA	Implémentation du Modèle (Scripts <code>CREATE TABLE</code>), Conception des Requêtes d'Agrégation (VUES comme <code>revenue_groupes</code>), Insertion des Données de Test (<code>INSERT</code>).

La collaboration s'est déroulée en deux phases principales :

1. **Conception et validation (étapes 1 et 2)** : Réflexion commune sur les associations et les cardinalités (notamment pour `follow`, `possede` et `historique`).
2. **Implémentation et test** : Séparation des tâches sur les scripts SQL. Une attention particulière a été portée à la table `record_historique` qui a remplacé la table `ecoute` jugée redondante, simplifiant le modèle final.